

COBOX

Logística local segura

Manual técnico

Arquitectura, código, base de datos, despliegue y mantenimiento de COBOX

Versión del documento: 1.0

Fecha: 02 de octubre de 2026

Repositorio: github.com/J-Vaz10/Special-project (rama main)

Commit analizado: 8a4b8dda18713af81a577ffeb6140fac4036a878

Audiencia: Equipo de desarrollo, calidad, asesores técnicos e inversionistas

Documentos relacionados: ERS COBOX (Rev. 1), DISEÑO COBOX (mockups), Manual de usuario



Manual técnico

Proyecto: COBOX

MATERIA:

Marcos de trabajo tecnologías de la información.

UNIVA

Ingeniería en sistemas y tecnologías de la información.

Contenido

1. Introducción	5
1.1 Propósito	5
1.2 Alcance	5
1.3 Equipo y división de trabajo	5
2. Visión general y arquitectura	6
2.1 Descripción funcional	6
2.2 Diagrama de arquitectura	6
2.3 Decisiones de diseño relevantes	6
3. Estructura del repositorio	8
4. Stack tecnológico	9
4.1 App móvil (`cobox/mobile`)	9
4.2 Servidor central (`cobox/supabase`)	9
4.3 App Android nativa (`cobox/proyecto 2`)	9
5. App móvil (Expo / React Native)	11
5.1 Arranque y navegación	11
5.2 Capa de servicios: `shipmentsService`	11
5.3 Modelo de dominio (`src/types/index.ts`)	12
5.4 Componentes de interfaz	13
5.5 Utilidades, tema y datos demo	14
5.6 Flujos principales	14
5.7 Permisos y configuración (`app.json`)	15
6. Servidor central (Supabase)	16
6.1 Esquema de base de datos	16
6.2 Almacenamiento de documentos KYC	16
6.3 Edge Functions	16
6.4 Variables de entorno	17
7. App Android nativa (prototipo del Comerciante)	18
8. Instalación, ejecución y verificación	19
8.1 App móvil	19
8.2 Servidor central (flujo sugerido con Supabase CLI)	19
8.3 App Android nativa	19
9. Integración pendiente con el backend	20
10. Trazabilidad con el ERS	21
Requisitos no funcionales destacados	22
11. Observaciones técnicas y deuda técnica	23
11.1 Seguridad	23
11.2 Consistencia entre esquema y aplicación	23

11.3 Aplicación móvil y prototipo Android.....	24
11.4 Documentación	24
12. Pruebas y calidad	25
13. Gestión del proyecto y documentación de negocio	26
13.1 Metodología	26
13.2 Plan de ejecución de 12 días.....	26
13.3 Presupuesto resumido (cotización del sprint)	26
13.4 Marco legal y societario	26
Anexo A. Glosario técnico	27
Anexo B. Referencias	28

1. Introducción

1.1 Propósito

Este manual documenta la implementación técnica de **COBOX**, plataforma móvil de logística colaborativa de última milla para el comercio local de Guadalajara y Zapopan que usa puntos de encuentro seguros **PUDO** (estaciones de MiBici y comercios afiliados). Describe cómo está organizado el repositorio, cómo funciona cada módulo, cómo ejecutarlo y qué falta para llevarlo a producción.

1.2 Alcance

El análisis cubre todo lo versionado en el repositorio: la app móvil (React Native / Expo), el servidor central (Supabase: migración SQL y dos Edge Functions), el prototipo Android nativo (Java) y los documentos de ingeniería (ERS, diseño, acta constitutiva y cotización).

Cómo leer este documento

Todo lo que se afirma como “implementado” se verificó leyendo el código fuente del commit indicado en la portada. Lo previsto en el ERS pero ausente en el código se marca como **pendiente**.

El capítulo 10 cruza los 30 requisitos funcionales del ERS con el código, y el capítulo 11 reúne las observaciones técnicas encontradas.

1.3 Equipo y división de trabajo

El README de [cobox/](#) define tres frentes de trabajo, que además corresponden a las etiquetas de GitHub previstas en el ERS (RNF-20):

Responsable	Frente	Carpeta / artefactos
Antonio	Servidor Central: base de datos, Auth, Storage, Edge Functions y APIs externas	cobox/supabase/
Jorge	Onboarding/KYC, registro de negocio, dashboard y crear envío (Comerciante)	cobox/proyecto_2/ (Android nativo)
Ángel	Bolsa de envíos, mapa/navegación, escáner de doble validación y app Comprador	cobox/mobile/

2. Visión general y arquitectura

2.1 Descripción funcional

Un **comerciante** crea un envío; un **repartidor** independiente lo toma de la bolsa, lo recoge (primera validación QR/PIN) y lo lleva a un **punto PUDO**, donde se confirma la entrega (segunda validación). El **comprador** rastrea el pedido y lo retira mostrando su QR o PIN. El servidor central debe garantizar la integridad de la secuencia de estados (RNF-10).

2.2 Diagrama de arquitectura

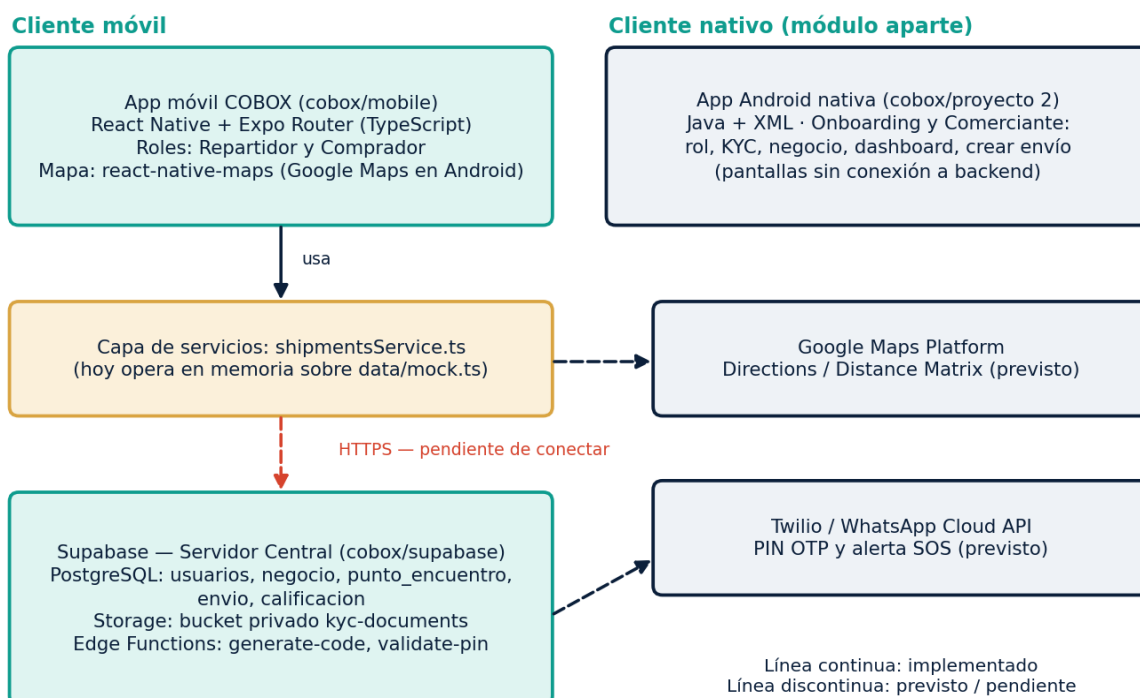


Figura 1. Arquitectura lógica de COBOX (estado actual del repositorio)

2.3 Decisiones de diseño relevantes

- **Una sola app móvil multi-rol (Expo Router):** los perfiles viven en el mismo proyecto con navegación por carpeta (/repartidor, /comprador).
- **BaaS en lugar de backend propio:** Supabase aporta PostgreSQL, Auth (JWT), Storage y Edge Functions (Deno). Cumple la arquitectura modular desacoplada de RNF-13.
- **Capa de servicios como frontera:** ninguna pantalla importa `data/mock` directamente; todo pasa por `shipmentsService.ts`. Conectar el backend real solo exige reescribir ese archivo.
- **Tipos como contrato:** `src/types/index.ts` modela las entidades del ERS y es el contrato que deben cumplir las Edge Functions.
- **Doble validación con respaldo por PIN:** cada transición exige QR o PIN; el PIN es el mecanismo de contingencia sin señal (RNF-09).

- **Secretos fuera del repositorio:** las llaves reales de Google Maps y Twilio van como secretos de Supabase o variables de entorno.

• 3. Estructura del repositorio

```

Special-project/
├── README.md                Descripción en una línea del proyecto
├── ERS COBOX.pdf            Especificación de requerimientos (42 págs.)
├── DISEÑO COBOX.pdf         9 mockups de interfaz
├── Acta_Constitutiva_COBOX_SAPI.docx  Constitución de la sociedad
├── Cotizacion_Presupuesto_COBOX_12Dias.docx  Plan y presupuesto del sprint
└── cobox/
    ├── README.md            Estructura y división de trabajo
    ├── .env.example         Variables de entorno (plantilla)
    ├── docs/                Copia de ERS y DISEÑO
    ├── mobile/              App Expo / React Native (TypeScript)
    │   ├── app.json · package.json · tsconfig.json
    │   ├── assets/          Íconos y splash
    │   └── src/
    │       ├── app/          Pantallas (Expo Router)
    │       ├── components/   14 componentes reutilizables
    │       ├── services/shipmentsService.ts
    │       ├── data/mock.ts  Datos de demostración
    │       ├── types/index.ts  Modelo de dominio
    │       └── theme/colors.ts · utils/geo.ts · utils/format.ts
    ├── proyecto 2/          Proyecto Android Studio (Java)
    │   └── app/src/main/{java,res,AndroidManifest.xml}
    └── supabase/
        ├── migrations/20240918000000_init_schema.sql
        └── functions/{generate-code,validate-pin}/index.ts
  
```

4. Stack tecnológico

4.1 App móvil (`cobox/mobile`)

Componente	Versión	Uso
Expo	~57.0.25	Plataforma y herramientas de build
React Native	0.86.3	Framework móvil multiplataforma
React	19.2.3	Interfaz declarativa
expo-router	~57.0.23	Navegación basada en archivos
TypeScript	~6.0.3	Tipado estricto (<code>strict: true</code>), alias <code>@/*</code> → <code>./src/*</code>
expo-camera	~57.0.5	Escáner de códigos QR
react-native-maps	1.27.2	Mapa (Google Maps en Android)
react-native-qrcode-svg + react-native-svg	^6.3.26 / 15.15.4	Generación del QR que muestra el comprador
@expo/vector-icons	^15.1.1	Iconografía (Ionicons)
react-native-gesture-handler / safe-area-context / screens	~2.32.0 / ~5.7.0 / ~4.26.0	Soporte de navegación y áreas seguras
ESLint + eslint-config-expo	^9.0.0 / ~57.0.2	Linter

4.2 Servidor central (`cobox/supabase`)

Componente	Detalle
Base de datos	PostgreSQL gestionado por Supabase (funciones <code>uuid_generate_v4()</code> y esquema <code>auth</code>)
Autenticación	Supabase Auth; <code>usuarios.id</code> referencia <code>auth.users</code>
Almacenamiento	Bucket privado <code>kyc-documents</code> con políticas por propietario
Edge Functions	TypeScript sobre Deno (<code>std@0.168.0</code>), <code>@supabase/supabase-js@2.7.1</code> vía <code>esm.sh</code>

4.3 App Android nativa (`cobox/proyecto 2`)

Componente	Detalle
Lenguaje	Java 11 (hay stubs <code>.kt</code> que remiten a las clases Java)
Plugin de Android (AGP)	9.4.1
SDK	<code>compileSdk 37</code> · <code>targetSdk 37</code> · <code>minSdk 24</code> (Android 7.0)
Librerías	AppCompat 1.6.1, Material 1.10.0, ConstraintLayout 2.1.4, Activity-KTX 1.8.0, Core-KTX 1.10.1

Componente	Detalle
Identificador	<code>com.example.proyecto especial</code> (versión 1.0, código 1)
Pruebas	JUnit 4.13.2, AndroidX JUnit 1.1.5, Espresso 3.5.1 (solo ejemplos plantilla)

Diferencias con el ERS

El ERS (§3.1.3) menciona React Native 0.74+ / Expo SDK 51, Android 8.0+ / iOS 14+ y PostgreSQL 15. El código usa Expo ~57 y React Native 0.86.3, y el prototipo Android admite desde Android 7.0 (`minSdk 24`). Se recomienda actualizar el ERS a las versiones reales.

5. App móvil (Expo / React Native)

5.1 Arranque y navegación

El punto de entrada es `expo-router/entry`. `src/app/_layout.tsx` envuelve la app en `GestureHandlerRootView` y `SafeAreaView`, fija la barra de estado clara y usa un `Stack` sin cabecera nativa: cada pantalla dibuja su propia cabecera con el componente `ScreenHeader`.

Ruta	Archivo	Pantalla	Requisitos ERS
/	<code>app/index.tsx</code>	Selector de demo (temporal): Repartidor o Comprador	RF-03 (sustituto)
/repartidor	<code>app/repartidor/index.tsx</code>	Paso 1 — Bolsa de envíos	RF-11 a RF-14
/repartidor/ruta/[id]	<code>ruta/[id].tsx</code>	Paso 2 — Mapa y navegación	RF-16, RF-19, RF-20
/repartidor/validar/[id]	<code>validar/[id].tsx</code>	Paso 3 — Escáner y doble validación	RF-22 a RF-25, RNF-09/10
/comprador	<code>app/comprador/index.tsx</code>	Mis pedidos	RF-09, RF-27
/comprador/seguimiento/[id]	<code>seguimiento/[id].tsx</code>	Paso 4 — Rastreo, código QR/PIN, calificación	RF-18, RF-26, RF-27, RF-29

Todas las pantallas dinámicas leen el parámetro `id` con `useLocalSearchParams`, cargan datos con `useFocusEffect` (se refrescan al volver a enfocar la pantalla) y cancelan las actualizaciones de estado si el componente se desmonta.

5.2 Capa de servicios: `shipmentsService`

Es la **única** capa que accede a datos. Hoy opera sobre un arreglo en memoria (`enviosDB`) sembrado desde `data/mock.ts`, con una latencia simulada de 350 ms (150 ms para puntos de encuentro) para imitar llamadas de red. Las firmas están diseñadas para mapear 1:1 con los endpoints reales.

Función	Comportamiento actual	Reemplazo previsto
<code>getBolsa(filtro, ubicación)</code>	Filtra envíos con <code>estado = 'creado'</code> , calcula la distancia Haversine desde la ubicación del repartidor (por defecto 20.6919, -103.3646) y ordena por cercanía , pago (mayor primero) o vehículo (bici/scooter primero).	Consulta a <code>ENVIO</code> + Distance Matrix API
<code>getEnvio(id)</code>	Busca un envío por ID.	Consulta por clave primaria
<code>getPuntosEncuentro()</code>	Devuelve los 6 puntos PUDO/MiBici de ejemplo.	Consulta a <code>PUNTO_ENCUENTRO</code> (RNF-15)
<code>getPedidosComprador(teléfono)</code>	Filtra por <code>compradorTelefono</code> y ordena por última actualización.	Consulta filtrada por el comprador autenticado
<code>aceptarEnvio(id, repartidor)</code>	Falla si el envío no existe o ya no está en <code>creado</code> ; si no, pasa a <code>asignado</code> y registra repartidor.	Edge Function con asignación atómica

Función	Comportamiento actual	Reemplazo previsto
<code>validarCodigo(envioId, tipo, método, valor)</code>	Valida QR o PIN, comprueba la fase y avanza el estado (ver 5.6).	Endpoint de validación QR/PIN (servidor)
<code>calificarEntrega(input)</code>	Devuelve la calificación con fecha; no persiste (TODO Antonio).	Insert en <code>CALIFICACION</code>
<code>reportarIncidente(input)</code>	Marca el envío como <code>incidente</code> y devuelve el reporte; no persiste el reporte.	Insert en tabla de incidentes (aún no existe)
<code>_reset()</code>	Restaura el estado inicial (solo pruebas/demo).	—

5.3 Modelo de dominio (`src/types/index.ts`)

Tipo	Definición
<code>RolUsuario</code>	<code>'comerciante' 'repartidor' 'comprador'</code>
<code>MedioTransporte</code>	<code>'a_pie' 'bicicleta' 'scooter'</code>
<code>TamanoPaquete</code>	<code>'pequeno' 'mediano' 'grande'</code>
<code>EstadoEnvio</code>	<code>'creado' 'asignado' 'en_transito' 'entregado' 'incidente'</code>
<code>TipoPuntoEncuentro</code>	<code>'comercio_pudo' 'estacion_mibici'</code>
<code>PuntoEncuentro</code>	<code>id, nombre, tipo, direccion, horario, referencias?, coordenadas, custodiaGratuitaHoras?, verificado</code>
<code>Envio</code>	<code>id (CBX-...), estado, comercioNombre, origen, destinoPudo, tamano, pesoKg, pagoMXN, esPagoVip?, distanciaKm, transporteRecomendado[], compradorNombre?, compradorTelefono, repartidorId?, repartidorNombre?, qrRecoleccion, qrEntrega, pin, creadoEn, actualizadoEn</code>
<code>TipoValidacion / MetodoValidacion</code>	<code>'recoleccion' 'entrega' / 'qr' 'pin'</code>
<code>ResultadoValidacion</code>	<code>{ ok, envio?, mensaje }</code>
<code>FiltroBolsa</code>	<code>'cercania' 'pago' 'vehiculo'</code>
<code>Calificacion</code>	<code>{ envioId, autor, estrellas 1-5, comentario?, creadoEn }</code>
<code>IncidenteReporte</code>	<code>{ envioId, reportadoPor, motivo, detalle?, creadoEn }</code>

Máquina de estados (RNF-10)

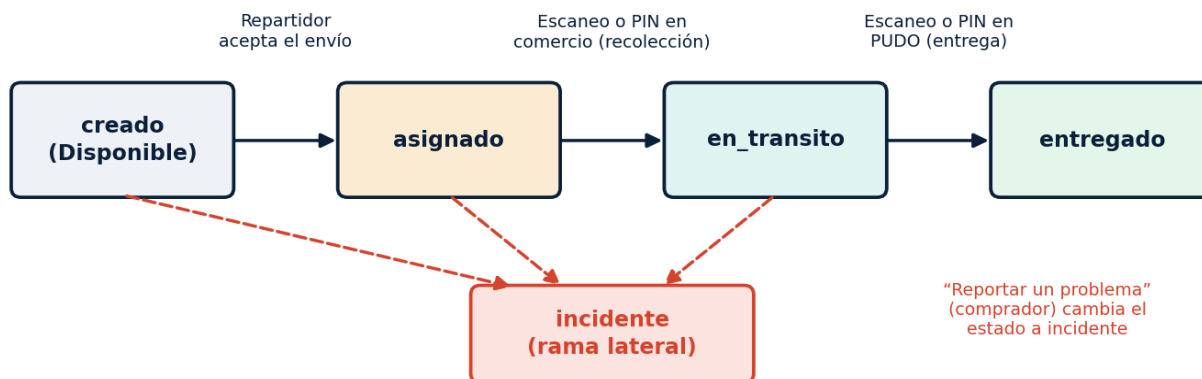


Figura 2. Estados del envío y transiciones permitidas

La secuencia válida es `creado` → `asignado` → `en_transito` → `entregado`; `incidente` es una rama lateral que `reportarIncidente` puede activar en cualquier punto. En la demo las reglas se aplican en el cliente; en producción **deben aplicarse en el servidor** para impedir saltos de estado.

5.4 Componentes de interfaz

Componente	Función
ScreenHeader	Cabecera azul marino con botón atrás, título, subtítulo y áreas <code>right</code> y <code>bottom</code> .
ShipmentCard	Tarjeta de la bolsa: distancia, etiqueta VIP, origen/destino, pago, tamaño, peso, transporte y botón Aceptar.
FilterBar	Filtros rápidos Más cerca / Mayor pago / Bici-Scooter (RF-12/13/14).
RouteMap	Mapa con Marker de origen y destino, pines de PUDO/MiBici, Polyline y leyenda. Proveedor Google en Android.
PudoInfoCard	Ficha operativa del PUDO: dirección, horario, distancia, custodia y referencias; escudo si está verificado (RF-20).
StepTracker	Indicador de 3 pasos Recoger → En tránsito → PUDO.
QRScannerModal	Escáner con <code>CameraView</code> (solo QR), solicitud de permiso y botón “Simular escaneo (dev)” en <code>__DEV__</code> .
PinBackupModal	Captura del PIN de 4 dígitos (solo números) para validar sin escanear (RNF-09).
CodigoEntregaCard	Muestra el QR (<code>react-native-qrcode-svg</code>) y el PIN con formato <code>CBX - 8974</code> .
RatingModal	Calificación de 1 a 5 estrellas (5 por defecto) con comentario opcional (RF-29).
ReportarProblemaModal	Motivos: Paquete dañado, No localizo al repartidor, Retraso prolongado, Otro.
SOSButton	Botón de emergencia con confirmación; punto de enganche para la alerta real por Twilio/WhatsApp.
Badge / EstadoBadge	Etiquetas de color; mapean cada <code>EstadoEnvio</code> a un texto y tono.
PrimaryButton	Botón con variantes (primary, secondary, ghost, danger), icono y estado de carga.

5.5 Utilidades, tema y datos demo

- ``utils/geo.ts``: `distanciaKm` (fórmula de Haversine, radio terrestre 6371 km), `construirRutaSimulada` (7 puntos con un leve desvío ondulado para dibujar la Polyline mientras no esté la Directions API) y `regionParaPuntos` (región del mapa con relleno 1.6× y mínimo 0.015°).
- ``utils/format.ts``: `formatoMXN`, `formatoPeso`, `etiquetaTamano`, `etiquetaTransporte`, `tiempoRelativo` y `estimarMinutos` (4.5 km/h a pie, 15 km/h bicicleta, 18 km/h monopatín; mínimo 2 min).
- ``theme/colors.ts``: paleta (azul marino `#0B1F3A`, verde azulado `#0E9C8E`, dorado `#D9A441`), espaciados y radios tomados de los mockups.
- ``data/mock.ts``: 6 puntos de encuentro (3 MiBici y 3 comercios, uno sin verificar), 1 envío activo (`CBX-240927-01`, PIN 8974) y 4 envíos en bolsa (`-02` a `-05`).

5.6 Flujos principales

Aceptar un envío

1. `BolsaEnviosScreen` llama a `getBolsa(filtro)` al enfocarse y al hacer pull-to-refresh.
2. Al tocar **Aceptar envío**, ejecuta `aceptarEnvio(id, REPARTIDOR_DEMO)`.
3. Si el estado ya no es `creado`, el servicio lanza “Este envío ya fue tomado por otro repartidor.” y la pantalla recarga la bolsa.
4. Si tiene éxito, navega a `/repartidor/ruta/{id}`.

Validar recolección o entrega (`validarCodigo``)

1. La pantalla deduce el tipo: `entrega` si el estado es `en_transito`; en otro caso `recoleccion`.
2. El valor (QR leído o PIN) se normaliza con `trim()` y mayúsculas.
3. Con método `pin` se compara contra `envio.pin`; con `qr`, contra `qrEntrega` o `qrRecoleccion` según el tipo.
4. Si no coincide: “El PIN no coincide con este envío.” o “El código QR no corresponde a este envío.”
5. Recolección exige estado `asignado`; entrega exige `en_transito` (mensajes “Este envío no está listo para recolección.” / “Aún falta confirmar la recolección en el comercio.”).
6. Si todo es válido, el estado pasa a `en_transito` (recolección) o `entregado` (entrega) y se actualiza `actualizadoEn`.

Formato del QR

El payload del QR es una cadena delimitada por barras verticales:

```
COBOX|<ID_ENVIO>|<RECOLECCION|ENTREGA>|<PIN>
Ejemplo: COBOX|CBX-240927-01|ENTREGA|8974
```

Calificación e incidente

`RatingModal` invoca `calificarEntrega({ envioId, autor: 'comprador', estrellas, comentario })`. `ReportarProblemaModal` invoca `reportarIncidente({ envioId, reportadoPor: 'comprador', motivo, detalle })`, que cambia el estado a `incidente`; la pantalla de seguimiento recarga y muestra “Reporte en revisión”. El botón SOS solo muestra confirmaciones locales.

5.7 Permisos y configuración (`app.json`)

Clave	Valor
<code>name / slug / scheme</code>	<code>cobox-repartidor-app / cobox-repartidor-app / coboxrepartidor</code>
<code>ios.bundleIdentifier</code>	<code>mx.cobox.repartidor</code>
<code>android.package</code>	<code>mx.cobox.repartidor</code>
Permisos Android	<code>CAMERA, ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION</code>
Textos iOS	<code>NSCameraUsageDescription, NSLocationWhenInUseUsageDescription</code>
Llaves de Google Maps	Placeholders <code>TU_API_KEY_DE_GOOGLE_MAPS_IOS / TU_API_KEY_DE_GOOGLE_MAPS_ANDROID</code> (aparecen en <code>ios.config</code> , <code>android.config</code> y en el plugin <code>react-native-maps</code>)
Orientación / tema	<code>Vertical; userInterfaceStyle: light</code>

6. Servidor central (Supabase)

6.1 Esquema de base de datos

La migración `20240918000000_init_schema.sql` crea cinco tablas en el esquema `public`:

Tabla	Columnas	Notas
<code>usuarios</code>	<code>id</code> UUID PK → <code>auth.users</code> , <code>nombre</code> , <code>email</code> UNIQUE, <code>telefono</code> , <code>rol</code> CHECK ('CLIENTE','REPARTIDOR','ADMIN'), <code>creado_en</code>	Perfil de aplicación ligado a Supabase Auth
<code>negocio</code>	<code>id</code> UUID PK, <code>nombre</code> , <code>direccion</code> , <code>creado_en</code>	Comercio local
<code>punto_encuentro</code>	<code>id</code> UUID PK, <code>latitud</code> , <code>longitud</code> , <code>direccion</code> , <code>descripcion</code>	Nodos PUDO / estaciones MiBici
<code>envio</code>	<code>id</code> UUID PK, <code>codigo_paquete</code> UNIQUE, <code>pin_dinamico</code> , <code>id_negocio</code> , <code>id_cliente</code> , <code>id_repartidor</code> , <code>id_punto_encuentro</code> , <code>estado</code> (default 'PENDIENTE'), <code>creado_en</code>	Entidad central; FKs a las otras cuatro
<code>calificacion</code>	<code>id</code> UUID PK, <code>id_envio</code> , <code>id_calificador</code> , <code>id_calificado</code> , <code>puntuacion</code> CHECK 1–5, <code>comentario</code> , <code>creado_en</code>	Retroalimentación (RF-29)

6.2 Almacenamiento de documentos KYC

La migración registra el bucket privado `kyc-documents` (`public = false`) y dos políticas sobre `storage.objects`: lectura (`SELECT`) y carga (`INSERT`) restringidas al propietario (`auth.uid() = owner`). Esto materializa RNF-06 (protección de INE/pasaporte/selfie conforme a la LFPDPPP).

6.3 Edge Functions

``generate-code``

Genera el identificador del envío y el PIN. No recibe parámetros relevantes ni escribe en la base de datos.

Aspecto	Detalle
Entrada	Cualquier petición HTTP
Salida 200	<code>{ "codigo_paquete": "CBX-AAMMDD-NN", "pin_dinamico": "NNNN" }</code>
Código	<code>CBX-</code> + fecha (AAMMDD) + sufijo aleatorio de 2 dígitos (00–99)
PIN	4 dígitos entre 1000 y 9999 (<code>Math.random</code>)
Error	400 con <code>{ "error": mensaje }</code>

``validate-pin``

Valida el PIN de un envío y, si es correcto, lo marca como entregado.

Aspecto	Detalle
Entrada (JSON)	<code>{ "codigo_paquete": string, "pin_dinamico": string }</code>

Aspecto	Detalle
400	<code>{ "error": "Faltan parametros" }</code> si falta alguno
404	<code>{ "error": "Envío no encontrado" }</code>
200	PIN correcto → <code>UPDATE envio SET estado = 'ENTREGADO'</code> y <code>{ success: true, message: "Validación exitosa, paquete entregado" }</code>
401	PIN incorrecto → <code>{ success: false, error: "PIN incorrecto" }</code>
Credenciales	Usa <code>SUPABASE_URL</code> y <code>SUPABASE_SERVICE_ROLE_KEY</code> del entorno

6.4 Variables de entorno

Variable	Para qué sirve	Dónde se configura
<code>SUPABASE_URL</code>	URL del proyecto Supabase	Entorno de las Edge Functions
<code>SUPABASE_ANON_KEY</code>	Llave pública para el cliente móvil	Cliente / entorno
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Llave con privilegios totales; solo servidor	Secretos de Supabase
<code>GOOGLE_MAPS_API_KEY</code>	Distance Matrix / Directions API	Secretos de Supabase
<code>TWILIO_ACCOUNT_SID</code> · <code>TWILIO_AUTH_TOKEN</code>	Credenciales de Twilio	Secretos de Supabase
<code>TWILIO_WHATSAPP_NUMBER</code>	Remitente de WhatsApp (por ejemplo <code>whatsapp:+14155238886</code>)	Secretos de Supabase

Seguridad de secretos

`cobox/.env.example` contiene solo valores de ejemplo. Los valores reales **no se suben al repositorio**: se cargan con `supabase secrets set NOMBRE=valor`. `SUPABASE_SERVICE_ROLE_KEY` nunca debe incluirse en la app móvil.

7. App Android nativa (prototipo del Comerciante)

El proyecto `cobox/proyecto 2` (Android Studio) implementa en Java el flujo de onboarding y las pantallas del comerciante. Son interfaces funcionales con validaciones básicas; **no se conectan a Supabase**.

Activity	Layout	Comportamiento
<code>RolSelectionActivity</code> (launcher)	<code>activity_rol_selection</code>	RadioGroup con Comerciante / Repartidor / Comprador y casilla de Aviso de Privacidad. Exige elegir rol y aceptar; pasa el rol por el extra <code>ROL_SELECCIONADO</code> a <code>KycActivity</code> .
<code>KycActivity</code>	<code>activity_kyc</code>	Botones de INE y selfie (solo muestran un Toast), campo RFC. Si el RFC está vacío muestra aviso; si no, "Identidad verificada" y abre <code>RegisterBusinessActivity</code> .
<code>RegisterBusinessActivity</code>	<code>activity_register_business</code>	Nombre, dirección, horario, teléfono y casilla PUDO. Exige nombre y dirección; abre <code>DashboardActivity</code> .
<code>DashboardActivity</code>	<code>activity_dashboard</code>	Muestra "Envíos Activos: 5" y "Saldo Disponible: \$350.00 MXN" (valores fijos del layout) y el botón + Nuevo envío .
<code>CreateShipmentActivity</code>	<code>activity_create_shipment</code>	Tamaño, peso y celular del comprador (obligatorios). Genera un número aleatorio de 4 dígitos y muestra "Envío creado... PIN: CBX-####"; no persiste.
<code>MainActivity</code>	<code>activity_main</code>	Plantilla de Android Studio ("Hello World"), declarada en el manifiesto pero fuera del flujo.

El flujo es lineal: `RolSelection` → `Kyc` → `RegisterBusiness` → `Dashboard` → `CreateShipment`. El manifiesto declara todas las activities; solo la primera es exportada (launcher).

Los archivos `.kt` de cada Activity son stubs de una línea ("migrado a Java") y pueden eliminarse.

8. Instalación, ejecución y verificación

8.1 App móvil

Requisitos: Node.js y npm instalados, un teléfono con Expo Go o un emulador Android/iOS.

```
cd cobox/mobile
npm install
npx expo start      # escanea el QR con Expo Go (Android) o abre un simulador
```

Expo Go vs. development build

`expo-camera` y `react-native-maps` son módulos nativos: **Expo Go solo sirve para probar la interfaz.**

Para escanear un QR real o ver Google Maps en Android hace falta un *development build*: `npx expo run:android` (o `npx expo run:ios`).

Antes del build, sustituye los placeholders de Google Maps en `app.json`.

Scripts disponibles en `package.json`: `start`, `android`, `ios`, `web` y `lint`. Comandos de verificación documentados por el equipo:

```
npx tsc --noEmit      # comprobación de tipos
npx expo lint         # lint
npx expo-doctor       # valida configuración y dependencias
```

8.2 Servidor central (flujo sugerido con Supabase CLI)

El repositorio no incluye `supabase/config.toml`; el README propone la convención del CLI (`supabase init`). Pasos típicos:

```
supabase init          # genera config.toml (si no existe)
supabase link --project-ref <ref>
supabase db push        # aplica migrations/20240918000000_init_schema.sql
supabase functions deploy generate-code
supabase functions deploy validate-pin
supabase secrets set GOOGLE_MAPS_API_KEY=... TWILIO_ACCOUNT_SID=... TWILIO_AUTH_TOKEN=...
```

Dependencia de la migración

La migración usa `uuid_generate_v4()`. Si el proyecto no tiene habilitada la extensión `uuid-oss`, actívala antes (`create extension if not exists "uuid-oss";`) o cambia a `gen_random_uuid()`.

8.3 App Android nativa

1. Abre la carpeta `cobox/proyecto 2` con Android Studio (requiere un AGP compatible con 9.4.1 y JDK 11 o superior).
2. Espera la sincronización de Gradle.
3. Ejecuta el módulo `app` en un emulador o dispositivo con Android 7.0 (API 24) o superior.

9. Integración pendiente con el backend

Para pasar de la demo a producción, la app móvil solo necesita reescribir `shipmentsService.ts`. Mapa de correspondencia propuesto por el equipo:

Función del servicio	Integración real
<code>getBolsa</code>	Consulta a <code>envio</code> (estado = creado) + Distance Matrix API para la distancia real
<code>aceptarEnvio</code>	Edge Function que asigne <code>id_repartidor</code> de forma atómica (evita doble asignación; RNF-11)
<code>validarCodigo</code>	Endpoint de validación QR/PIN; debe aplicar la máquina de estados en el servidor
<code>getPuntosEncuentro</code>	Consulta a <code>punto_encuentro</code> (alta dinámica de nodos sin recompilar; RNF-15)
<code>getPedidosComprador</code>	Consulta a <code>envio</code> filtrada por el comprador autenticado
<code>calificarEntrega / reportarIncidente</code>	Inserts en <code>calificacion</code> y en una tabla de incidentes (por crear)
<code>SOSButton</code>	Alerta SMS/WhatsApp vía Twilio disparada desde el servidor
<code>RouteMap</code>	Reemplazar <code>construirRutaSimulada</code> por la polilínea de la Directions API
Usuario demo (<code>REPARTIDOR_DEMO</code> , <code>COMPRADOR_DEMO_TELEFONO</code>)	Usuario autenticado con Supabase Auth

Orden de trabajo recomendado:

1. Ajustar el esquema SQL al modelo de dominio (ver capítulo 11) y habilitar Row Level Security.
2. Implementar Supabase Auth y el selector de rol real; eliminar los usuarios demo.
3. Conectar `getBolsa`, `getPuntosEncuentro`, `getPedidosComprador` y `aceptarEnvio`.
4. Mover la validación de QR/PIN y las transiciones de estado al servidor.
5. Integrar Google Maps (rutas/distancias) y Twilio (OTP, alertas, SOS).
6. Añadir la cola de peticiones offline para validaciones por PIN (ERS §2.1.8.3).
7. Conectar el flujo del comerciante (Android nativo o migración a Expo) al mismo backend.

10. Trazabilidad con el ERS

Estado de cada requisito funcional según la revisión del código:

Significado	
Demo	Implementado y funcional con datos de ejemplo en la app móvil.
Parcial	Existe la interfaz, el esquema o una pieza del servidor, pero falta integración o persistencia.
Pendiente	Sin implementación en el repositorio.

RF	Requisito	Estado	Evidencia en el repositorio
RF-01	Registro de usuarios	Parcial	Tabla <code>usuarios</code> ; selección de rol en Android
RF-02	Control de sesiones	Pendiente	Supabase Auth previsto, sin pantallas de login
RF-03	Adaptación por perfil	Parcial	<code>RolSelectionActivity</code> y selector demo de la app Expo
RF-04	Verificación de identidad	Parcial	<code>KycActivity</code> (sin captura real); bucket <code>kyc-documents</code>
RF-05	Datos del comercio	Parcial	<code>RegisterBusinessActivity</code> ; tabla <code>negocio</code>
RF-06	Dimensiones del paquete	Parcial	<code>CreateShipmentActivity</code> (tamaño y peso); sin persistencia
RF-07	Direcciones del envío	Pendiente	No hay campos de origen/destino en la pantalla Android
RF-08	Ticket de rastreo	Parcial	Función <code>generate-code</code> ; IDs demo en <code>mock.ts</code>
RF-09	Listado de envíos	Parcial	<code>getPedidosComprador</code> /Mis pedidos (comprador); falta vista del comerciante
RF-10	Monitoreo de estado	Parcial	Seguimiento del comprador; falta vista del comerciante
RF-11	Visualización de ofertas	Demo	<code>BolsaEnviosScreen</code> + <code>getBolsa</code>
RF-12	Filtrado geográfico	Demo	Filtro “Más cerca” (Haversine)
RF-13	Filtrado por tarifa	Demo	Filtro “Mayor pago”
RF-14	Vehículo recomendado	Demo	Filtro “Bici / Scooter” y etiquetas de transporte
RF-15	Toma de pedidos	Demo	<code>aceptarEnvio</code> (en memoria)
RF-16	Despliegue de trayecto	Parcial	<code>RouteMap</code> con ruta simulada; Directions API pendiente
RF-17	Asignación de punto seguro	Pendiente	Sin lógica de sugerencia (solo en el mockup)
RF-18	Notificación de destino PUDO	Demo	Ficha del PUDO en seguimiento; sin notificación push
RF-19	Visualización de nodos PUDO	Demo	Pines de PUDO/MiBici en <code>RouteMap</code>
RF-20	Ficha operativa PUDO	Demo	<code>PudoInfoCard</code>
RF-21	Registro de nodos públicos	Parcial	Tabla <code>punto_encuentro</code> ; datos demo en <code>mock.ts</code>
RF-22	Código único de seguridad	Parcial	QR en <code>CodigoEntregaCard</code> ; PIN en <code>generate-code</code>
RF-23	Escaneo en comercio	Demo	Validación <code>recoleccion</code> con QR/PIN
RF-24	Escaneo al comprador	Demo	Validación <code>entrega</code> con QR/PIN

RF	Requisito	Estado	Evidencia en el repositorio
RF-25	Transición de estados	Demo	<code>validarCodigo</code> (cliente); <code>validate-pin</code> (servidor, parcial)
RF-26	Alertas de arribo	Pendiente	Sin notificaciones push ni Twilio integrado
RF-27	Detalle del pedido	Demo	Pantalla de seguimiento del comprador
RF-28	Registro de actividad	Pendiente	Sin historial por repartidor
RF-29	Retroalimentación	Parcial	<code>RatingModal</code> (UI); no persiste en <code>calificacion</code>
RF-30	Lógica de negocio central	Parcial	Esquema SQL y dos Edge Functions

Requisitos no funcionales destacados

RNF	Requisito	Estado actual
RNF-04	Cifrado en comunicaciones (HTTPS/TLS)	Cubierto por Supabase; la app aún no consume el backend
RNF-06	Protección de documentos KYC	Bucket privado con políticas por propietario (esquema listo)
RNF-07	Control de acceso por perfil	Pendiente: sin RLS ni autenticación
RNF-09	Operación con redes inestables	Parcial: PIN de respaldo manual; falta cola offline
RNF-10	Integridad de estados	Demo en cliente; pendiente en servidor
RNF-11	Prevención de duplicados	Pendiente (asignación atómica y claves de idempotencia)
RNF-13	Arquitectura modular	Cumplido: capa de servicios desacoplada de la UI
RNF-15	Alta dinámica de nodos PUDO	Parcial: tabla lista; la app aún lee datos locales
RNF-17/19	Pantallas adaptables y mapas consistentes	Parcial: estilos adaptables; mapa pendiente de llaves
RNF-20	Gestión ágil con etiquetas en GitHub	El repositorio tiene Issues habilitados (6 abiertos)

11. Observaciones técnicas y deuda técnica

Hallazgos de la revisión, ordenados por impacto. Ninguno impide la demo; sí conviene resolverlos antes de operar con usuarios reales.

11.1 Seguridad

#	Hallazgo	Recomendación
S1	La migración crea las tablas pero no habilita Row Level Security . Con la llave anónima en el cliente, las tablas quedarían expuestas por la API REST.	<code>ALTER TABLE ... ENABLE ROW LEVEL SECURITY</code> y políticas por rol/propietario (RNF-07).
S2	<code>validate-pin</code> usa la llave de servicio sin autenticar al llamante, no verifica rol ni estado previo y marca <code>ENTREGADO</code> directamente.	Exigir JWT, validar que el llamante es el repartidor asignado y aplicar la máquina de estados (recolección → entrega).
S3	El PIN es de 4 dígitos (10 000 combinaciones) y no hay límite de intentos.	Limitar intentos por envío, bloquear temporalmente y registrar en bitácora (RNF-08).
S4	El QR es predecible (<code>COBOX id tipo pin</code>) y no tiene firma ni vencimiento, aunque el ERS pide mecanismos criptográficos.	Firmar el payload (HMAC/JWT) con expiración y validarlo en el servidor.
S5	<code>generate-code</code> usa <code>Math.random</code> , que no es criptográficamente seguro.	Usar <code>crypto.getRandomValues</code> para PIN y sufijos.
S6	Un mismo PIN de 4 dígitos sirve para validar tanto la recolección como la entrega, y el comprador lo conoce; quien lo tenga podría completar ambos pasos.	Separar PIN de recolección y de entrega, y mostrar a cada actor solo el suyo.

11.2 Consistencia entre esquema y aplicación

#	Hallazgo	Recomendación
D1	<code>usuarios.rol</code> admite <code>CLIENTE/REPARTIDOR/ADMIN</code> ; la app usa <code>comerciante/repartidor/comprador</code> .	Unificar los valores del rol.
D2	<code>envio.estado</code> tiene valor por defecto <code>PENDIENTE</code> y <code>validate-pin</code> escribe <code>ENTREGADO</code> ; la app usa <code>creado/asignado/en_transito/entregado/incidente</code> . Además <code>estado</code> no tiene restricción <code>CHECK</code> .	Adoptar un único conjunto de estados y agregar un <code>CHECK</code> o tipo <code>ENUM</code> .
D3	Faltan columnas que la app necesita: en <code>negocio</code> (teléfono, horario, ubicación, bandera PUDO, propietario), en <code>punto_encuentro</code> (nombre, tipo, horario, verificado, custodia) y en <code>envio</code> (tamaño, peso, pago, distancia, transporte).	Ampliar el esquema con una nueva migración.
D4	No existen tablas para incidentes, estado de verificación KYC ni historial de actividad del repartidor (RF-28).	Crear <code>incidente</code> , <code>kyc</code> y registro de eventos del envío.
D5	<code>generate-code</code> no persiste ni garantiza unicidad: el sufijo de 2 dígitos produce colisiones frente a la restricción <code>UNIQUE</code> de <code>codigo_paquete</code> .	Generar el código dentro de una transacción con secuencia diaria y reintento ante colisión.

11.3 Aplicación móvil y prototipo Android

#	Hallazgo	Recomendación
A1	No hay autenticación: <code>REPARTIDOR_DEMO</code> y <code>COMPRADOR_DEMO_TELEFONO</code> están fijos en el código.	Integrar Supabase Auth y leer el usuario de la sesión.
A2	Los datos viven en memoria y se pierden al reiniciar; calificaciones y reportes no se guardan.	Persistir en Supabase (capítulo 9).
A3	No existe la cola de peticiones offline prevista en el ERS §2.1.8.3; el PIN se valida en línea.	Encolar validaciones localmente y reintentar con idempotencia (RNF-11).
A4	La app Expo contiene solo los perfiles Repartidor y Comprador; el Comerciante vive en un proyecto Android aparte con otro stack.	Decidir si se migra el flujo del comerciante a Expo para tener una sola base de código (RNF-13/16).
A5	<code>KycActivity</code> exige RFC/CURP aunque el diseño lo marca opcional; la cámara y la selfie no están implementadas; el aviso de privacidad es solo una casilla (el ERS pide un aviso accesible, derechos ARCO y consentimiento explícito).	Alinear con el mockup y con §3.5.1 del ERS.
A6	<code>DashboardActivity</code> muestra valores fijos; <code>CreateShipmentActivity</code> no guarda ni muestra el QR.	Conectar con el backend.
A7	Residuos: <code>MainActivity</code> (“Hello World”) y stubs <code>.kt</code> duplicados; carpeta <code>proyecto 2</code> con espacio y <code>applicationId</code> de plantilla (<code>com.example</code>).	Limpiar y renombrar; usar un <code>applicationId</code> propio (la app Expo usa <code>mx.cobox.repartidor</code>).
A8	El nombre del paquete Expo (<code>cobox-repartidor-app</code>) no refleja que incluye el perfil Comprador.	Renombrar a <code>cobox</code> en <code>app.json</code> y <code>package.json</code> .

11.4 Documentación

- El README de `supabase/` nombra las funciones `generar-codigo-envio` y `validar-qr-pin`; en el código se llaman `generate-code` y `validate-pin`.
- El README raíz tiene una sola línea; este manual puede servir de base para ampliarlo.
- El ERS deja las secciones 5 (Código) y 6 (Pruebas) en “TBD”.
- El ERS indica versiones de Expo/React Native/PostgreSQL distintas a las del código (ver capítulo 4).

12. Pruebas y calidad

Área	Estado
App móvil	Sin pruebas unitarias ni de integración. El equipo reporta que <code>tsc --noEmit</code> , <code>expo lint</code> y <code>expo-doctor</code> pasan sin errores.
Android nativo	Solo <code>ExampleUnitTest</code> ($2 + 2 = 4$) y <code>ExampleInstrumentedTest</code> de la plantilla.
Edge Functions	Sin pruebas automatizadas.
Pruebas manuales	El plan del sprint contempla pruebas de campo con 3 comercios reales en Zapopan (días 9–10) y pruebas de carga (día 11).

Casos de prueba prioritarios que se recomienda automatizar:

- `getBolsa`: orden por cercanía, pago y vehículo; exclusión de envíos no `creado`.
- `aceptarEnvio`: doble aceptación concurrente (el segundo intento debe fallar).
- `validarCodigo`: QR/PIN correcto e incorrecto, recolección sin estado `asignado`, entrega sin recolección previa.
- `distanciaKm`: distancias conocidas entre puntos del Centro de Guadalajara.
- `validate-pin`: 400, 404, 200 y 401.

13. Gestión del proyecto y documentación de negocio

13.1 Metodología

Desarrollo ágil con GitHub Projects, Issues y Milestones, usando etiquetas diferenciadas para la App Comerciante, la App Repartidor/Comprador y el Servidor Central (ERS §2.4, RNF-20). El repositorio es público y tiene seis Issues abiertos.

13.2 Plan de ejecución de 12 días

Fase	Actividades y entregables
Días 1–2	Historias de usuario, diseño UI/UX en Figma y configuración del backend BaaS.
Días 3–5	Módulo remitente/usuario: registro, geolocalización de puntos, solicitud de envío y códigos QR/PIN.
Días 6–8	Módulo punto afiliado: escaneo en mostrador y verificación de PIN.
Días 9–10	Pruebas de campo de extremo a extremo con 3 comercios reales en Zapopan.
Día 11	Code freeze, pruebas de carga, manuales operativos y carpeta corporativa.
Día 12	Demo Day: demostración con un paquete real y entrega formal.

13.3 Presupuesto resumido (cotización del sprint)

Capítulo de gasto	Monto (MXN)	%
Capital humano (5 integrantes)	\$60,480.00	77.0 %
Infraestructura tecnológica, cloud y APIs	\$7,100.00	9.0 %
Marco jurídico y protección de marca	\$4,700.00	6.0 %
Operación y validación piloto en Zapopan	\$6,300.00	8.0 %
Gran total del sprint (12 días)	\$78,580.00	100 %

13.4 Marco legal y societario

- **Sociedad:** COBOX, S.A.P.I. de C.V., con domicilio en Zapopan, Jalisco; capital mínimo fijo de \$60,000.00 MXN en 6,000 acciones de \$10.00, repartidas en partes iguales entre seis socios (cinco fundadores y un asesor/cliente). El acta conserva campos pendientes entre corchetes (apellidos y datos del asesor y del comisario).
- **Propiedad intelectual:** todo el software, código, diseños y marcas son propiedad exclusiva de la sociedad (art. 8). Las acciones tienen un esquema de *vesting* de 3 años (art. 9).
- **Normativa aplicable al producto:** LFPDPPP (aviso de privacidad y derechos ARCO), Ley Federal de Protección al Consumidor (art. 76 Bis 1) y Ley de Movilidad, Seguridad Vial y Transporte del Estado de Jalisco.

Anexo A. Glosario técnico

Término	Definición
BaaS	Backend as a Service: plataforma que ofrece base de datos, autenticación y almacenamiento (Supabase).
Edge Function	Función serverless en TypeScript/Deno desplegada en Supabase.
Expo Router	Sistema de navegación donde cada archivo en <code>src/app</code> es una ruta.
Development build	Compilación nativa de la app con módulos como cámara y mapas, necesaria fuera de Expo Go.
Haversine	Fórmula para calcular la distancia sobre una esfera entre dos coordenadas.
PUDO	Pick-up and Drop-off: punto físico de recolección y entrega.
RLS	Row Level Security: reglas de PostgreSQL que limitan qué filas puede ver o modificar cada usuario.
ERS	Especificación de Requerimientos de Software (IEEE 29148).

Anexo B. Referencias

- ERS COBOX, Revisión 1 — [ERS COBOX.pdf](#) (42 páginas, 30 RF y 20 RNF).
- DISEÑO COBOX — [DISEÑO COBOX.pdf](#) (9 pantallas).
- [cobox/README.md](#), [cobox/mobile/README.md](#) y [cobox/supabase/README.md](#).
- IEEE 29148-2018 — Requirements engineering.
- Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP).
- Ley Federal de Protección al Consumidor, art. 76 Bis 1.
- Ley de Movilidad, Seguridad Vial y Transporte del Estado de Jalisco.